

Relatório Engenharia de Software

Miguel Correia

Universidade de Évora, Évora, Portugal
m57609@alunos.uevora.pt

Abstract. O desenvolvimento de software contemporâneo enfrenta desafios complexos que exigem abordagens eficientes e escaláveis. Entre essas abordagens, destacam-se a reutilização de software e o desenvolvimento baseado em serviços, que proporcionam flexibilidade e rapidez. Este artigo discute a reutilização de software, a arquitetura orientada a serviços, a utilização de containers, e aborda as práticas e métodos de teste de software, com ênfase nos sistemas baseados em serviços.

Keywords: Reutilização · Serviços · Testes.

1 Introdução

No cenário atual do desenvolvimento de software, as organizações enfrentam desafios cada vez mais complexos que exigem soluções eficientes e escaláveis. Duas abordagens fundamentais para enfrentar estes desafios são a reutilização de software e o desenvolvimento baseado em serviços. A reutilização de software envolve o aproveitamento de componentes já existentes, o que pode aumentar significativamente a eficiência e a qualidade dos projetos. Paralelamente, a Arquitetura Orientada a Serviços (SOA) emerge como um paradigma que promove a flexibilidade e escalabilidade, permitindo que funções de software sejam disponibilizadas como serviços independentes e interoperáveis.

Além disso, a utilização de containers, como Docker e Kubernetes, tem se tornado uma prática essencial para garantir a portabilidade e o isolamento de aplicações, facilitando a sua gestão e escalabilidade. No entanto, com a crescente complexidade dos sistemas, é crucial adotar práticas robustas de teste de software. Este artigo foca especialmente nos desafios e abordagens para testar sistemas baseados em serviços, incluindo testes de contrato, de integração e end-to-end.

Por fim, o artigo apresenta métodos para garantir a consistência dos serviços ao longo do seu ciclo de vida, através de práticas de versionamento, monitorização e logging. Ao adotar estas práticas, as organizações podem melhorar significativamente a eficiência, flexibilidade e qualidade dos seus processos de desenvolvimento de software. Este artigo pretende oferecer uma visão abrangente destas abordagens, destacando a sua importância e fornecendo diretrizes práticas para a sua implementação eficaz.

2 Reutilização de Software

A reutilização de código, ou reutilização de software, é o uso de software existente ou do conhecimento de software existente para criar software novo.

Esta prática tem sido feita desde o princípio da programação. Os programadores reutilizam seções de código, modelos(templates), funções e procedimentos.

Uma vantagem é a poupança de tempo pois está-se a construir software novo com partes já existentes ao invés de construí-lo de raiz, o que leva muito mais tempo. Outra vantagem é que as soluções previamente desenvolvidas poderão ser aproveitadas e implementadas num novo contexto.

Desenvolver sistemas é tipicamente caro e lento. Existe sempre uma busca de redução de custos e de ganhos incomparáveis em qualidade de produto, como redução de tempo de desenvolvimento fazem com que o produto final fique aquém do desejado. Aqui a reutilização de software é uma possível solução para lidar com este processo.

A reutilização de software baseia-se no uso de conceitos, produtos ou soluções previamente elaboradas ou adquiridas para criação de um novo software, para melhorar a qualidade e a produtividade. Reutilizam-se partes de um sistema desenvolvido anteriormente como especificações, módulos de um projeto, arquitetura e código fonte. A motivação principal é o aumento dos níveis de qualidade e produtividade no desenvolvimento de software.

Pode ser visto como falta de originalidade, mas durante o desenvolvimento do software pode resultar no aumento de produtividade e numa redução considerável no tempo de execução das tarefas. É mais fácil reutilizar componentes numa aplicação que seja do mesmo domínio da que foi originalmente desenvolvida por refletirem os mesmos conceitos e relações.

3 Desenvolvimento de Software Baseado em Serviços (SOA)

SOA é um método de desenvolvimento de software que usa componentes de software chamados de serviços para criar aplicações de negócios. Cada serviço fornece um recurso de negócios, e todos também pode comunicar entre si em plataformas diferentes e em linguagens diferentes. O SOA reutiliza os serviços em sistemas diferentes ou combina vários serviços independentes para realizar tarefas complexas.

Por exemplo, vários processos de negócios em uma organização exigem a funcionalidade de autenticação de usuários. Em vez de reescrever o código de autenticação para todos os processos de negócios, pode-se criar um único serviço de autenticação e reutilizá-lo para todas as aplicações.

Os benefícios desta arquitetura são maior rapidez na entrada da aplicação no mercado, manutenção eficiente, maior adaptabilidade e redução da redundância.

Maior rapidez na entrada no mercado pois são reutilizados serviços já existentes em diferentes processos de negócios de modo a poupar tempo e economizar custos. Com SOA as aplicações são estruturadas mais rapidamente do que seriam escrevendo código do zero e fazendo integrações do zero.

Manutenção eficiente pois é mais fácil criar, atualizar e depurar pequenos serviços do que grandes blocos de código em aplicações monolíticas. A modificação de qualquer serviço na SOA não afeta a funcionalidade geral do processo de negócios.

Maior adaptabilidade pois a SOA é mais adaptável aos avanços da tecnologia. Pode-se modernizar as aplicações de maneira eficiente e económica. Por exemplo, as organizações de saúde podem usar a funcionalidade de sistemas de EHR mais antigos em aplicações baseadas na nuvem mais recentes.

Redução da redundância pois as funcionalidades reutilizáveis são desenvolvidas apenas uma vez e podem ser partilhadas entre diferentes plataformas e serviços. A redução da redundância é o que permite todos benefícios anteriormente citados.

Uma plataforma popular que usa esta arquitetura é a Amazon Web Services (AWS).

As características de SOA no AWS são:

- **Autonomia dos Serviços:** Cada serviço da AWS é independente e pode ser utilizado separadamente ou em combinação com outros serviços, dependendo das necessidades específicas do usuário.
- **Interfaces Bem Definidas:** Os serviços da AWS possuem APIs claras e bem documentadas, permitindo fácil integração e utilização.
- **Reutilização e Composição:** Os serviços podem ser reutilizados em diferentes soluções e contextos, promovendo a flexibilidade e a modularidade.
- **Interoperabilidade:** Os serviços da AWS são projetados para funcionar bem juntos, bem como com sistemas externos, facilitando a integração e a interoperabilidade.

No entanto existem desafios na implementação da SOA, tais como a integração das componentes reutilizadas em novos sistemas, pois pode ser muito complexa, e a compatibilidade dessas mesmas componentes entre si e com o novo sistema.

4 Utilização de containers

Containers são unidades padronizadas de software que empacotam o código e todas as suas dependências, garantindo que ele seja executado de maneira consistente em qualquer ambiente.

A reutilização de containers tem grandes benefícios, tais como:

Portabilidade : Containers garantem que o software funcione de maneira consistente em diferentes ambientes, desde o desenvolvimento até à conclusão. Esta portabilidade é essencial para a reutilização de software, pois assegura que os

componentes possam ser movidos e integrados facilmente entre diferentes sistemas e plataformas.

Isolamento: Cada container é isolado, o que reduz conflitos entre dependências de diferentes aplicações.

Escalabilidade: Facilita o escalonamento de aplicações distribuídas.

Algumas ferramentas populares são Docker e Kubernetes.

Docker é uma plataforma utilizada para criar, implantar e gerir containers. Permite que os desenvolvedores empacotem aplicações e as suas dependências em containers, garantindo que o software funcione de maneira consistente em qualquer ambiente.

Kubernetes é um sistema de orquestração de containers que automatiza a implantação, escalonamento e operações de containers. Permite a gestão eficiente de clusters de containers, facilitando a reutilização e escalabilidade de componentes de software.

5 Testes de Software

Testes são essenciais para garantir a qualidade, funcionalidade e desempenho do software. Ajudam a identificar e corrigir defeitos antes do lançamento.

Existem tipos diferentes de testes de software: testes unitários, testes de integração, testes funcionais, testes de ponta a ponta, testes de aceitação, testes de desempenho e smoke tests.

5.1 Testes unitários

Testes unitários são feitos em um nível muito baixo, próximo ao código-fonte do aplicativo. Eles consistem em testar métodos e funções individuais de classes, componentes ou módulos usados pelo software. Testes de unidade, em geral, têm custo baixo para automatizar e podem ser executados com rapidez por um servidor de integração contínua.

5.2 Testes de integração

Testes de integração verificam se diferentes módulos ou serviços usados pelo seu aplicativo funcionam bem juntos. Por exemplo, pode ser testar a interação com o banco de dados ou garantir que os microserviços funcionem juntos conforme o esperado. A execução desses tipos de testes tem um custo maior, uma vez que exigem que várias partes do aplicativo estejam ativas e em execução.

5.3 Testes funcionais

Os testes funcionais têm como foco os requisitos de negócios de uma aplicação. Eles só verificam a saída de uma ação e não verificam os estados intermediários do sistema ao executar essa ação.

Às vezes há uma confusão entre testes de integração e testes funcionais, Testes End-to-End (E2E) são uma metodologia de teste de software que verifica a funcionalidade e o comportamento de uma aplicação desde o início até ao fim, replicando o percurso real de um utilizador. O objetivo é garantir que todos os componentes do sistema funcionem corretamente e de forma integrada, desde a interface do utilizador até ao backend e bases de dados. uma vez que ambos exigem vários componentes para interagirem entre si. A diferença é que um teste de integração pode simplesmente verificar que você pode consultar o banco de dados, enquanto um teste funcional esperaria obter um valor específico do banco de dados conforme definido pelos requisitos do produto.

5.4 Testes Ponta a ponta

Testes End-to-End (E2E) são uma metodologia de teste de software que verifica a funcionalidade e o comportamento de uma aplicação desde o início até ao fim, replicando o percurso real de um utilizador. O objetivo é garantir que todos os componentes do sistema funcionem corretamente e de forma integrada, desde a interface do utilizador até ao backend e bases de dados.

5.5 Testes de Validação

Testes de validação são um conjunto de atividades realizadas para assegurar que um software atende às necessidades e expectativas do utilizador final e aos requisitos especificados. O objetivo principal é garantir que o produto desenvolvido está em conformidade com o seu propósito e se comporta conforme esperado em condições reais de uso.

Os testes de validação garantem que todos os requisitos especificados, tanto funcionais como não-funcionais, foram implementados corretamente, aAsseguram que o produto final cumpre as expectativas e necessidades dos utilizadores, aumentando a satisfação do cliente, contribuem para a entrega de um produto de alta qualidade, minimizando defeitos e problemas de usabilidade e em indústrias regulamentadas, os testes de validação ajudam a garantir que o software cumpre os padrões e regulamentos aplicáveis.

6 Testes de Software e Sistemas baseados em Serviços

Sistemas baseados em serviços são uma abordagem arquitetónica onde a funcionalidade da aplicação é dividida em serviços independentes que se comunicam através de interfaces bem definidas. Permite maior flexibilidade, escalabilidade e manutenção das aplicações. No entanto, a natureza distribuída e a interdependência desses serviços apresentam desafios únicos para os testes de software.

Um dos desafios dos testes em sistemas baseados em serviços é a complexidade de integração. A interação entre múltiplos serviços pode levar a problemas de integração que são difíceis de identificar e resolver. Cada serviço pode ter dependências diferentes e ser desenvolvido por equipes distintas, aumentando a complexidade dos testes.

Outro desafio são os ambientes distribuídos. Testar em ambientes distribuídos exige a simulação de múltiplos serviços funcionando simultaneamente, o que pode ser tecnicamente desafiador e exigir recursos significativos.

6.1 Abordagens

Existem vários tipos de abordagem:

Testes Unitários

Focam-se em testar componentes individuais dos serviços, garantindo que cada parte do código funciona conforme esperado. Utilizam stubs e mocks para isolar o serviço em teste das suas dependências externas.

Testes de Integração

Verificam se os diferentes serviços funcionam corretamente quando integrados. Testam as interações entre serviços, incluindo chamadas de API, transferência de dados, e respostas adequadas. Utilizam ambientes de teste que replicam a infraestrutura de produção o mais fielmente possível.

Testes de Contrato

Asseguram que os serviços respeitam os contratos de interface definidos, isto é, que a comunicação entre serviços segue as especificações acordadas.

Utilizam ferramentas como Pact para criar contratos que são verificados tanto pelo consumidor quanto pelo provedor do serviço.

Testes End-to-End (E2E)

Testam fluxos de trabalho completos que envolvem múltiplos serviços, desde o início até ao fim. Simulam cenários reais de utilização para garantir que a aplicação funciona corretamente do ponto de vista do utilizador final.

Testes de Desempenho e Escalabilidade

Avaliam como os serviços se comportam sob diferentes cargas de trabalho.

Utilizam ferramentas como JMeter ou Gatling para simular tráfego e medir tempos de resposta, throughput e outros indicadores de desempenho.

Testes de Resiliência

Testam como os serviços se comportam em situações de falha, como perda de conectividade ou falhas de hardware. Utilizam técnicas de chaos engineering, como as oferecidas pelo Chaos Monkey, para introduzir falhas aleatórias e verificar a capacidade de recuperação dos serviços.

Testes de Segurança Verificam a robustez dos serviços contra ameaças e vulnerabilidades de segurança. Incluem testes de penetração, análise de vulnerabilidades e verificação de conformidade com padrões de segurança.

6.2 Ferramentas populares

As ferramentas mais populares para testar sistemas baseados em serviços são:

Postman

É utilizado para testar APIs, permitindo a criação, execução e gestão de testes de API de maneira fácil e eficiente.

SoapUI

Ferramenta de teste de APIs SOAP e REST, oferecendo capacidades avançadas de criação e execução de testes.

WireMock

Utilizado para criar mocks de serviços HTTP, permitindo a simulação de interações com serviços externos.

Pact

Ferramenta de teste de contrato que facilita a verificação de contratos entre consumidores e provedores de serviços.

7 Métodos para Garantir a Consistência dos Serviços ao Longo do Seu Ciclo de Vida

Versionamento de APIs

-Mantém diferentes versões de APIs para garantir que mudanças não quebrem a compatibilidade com clientes existentes.

Monitorização Contínua

-Implementação de ferramentas de monitorização para observar a saúde e desempenho dos serviços em tempo real.

Logging e Tracing

-Utilização de logs detalhados e tracing distribuído para diagnosticar problemas e rastrear o fluxo de dados através dos serviços.

Automatização de Testes

-Integração de testes automatizados nos pipelines de CI/CD para garantir que todas as alterações são verificadas antes de serem implementadas em produção.

[4],[1].[3],[2], [6],[5]

References

- [1] Amazon. *O que é SOA (arquitetura orientada a serviços)?* URL: <https://aws.amazon.com/pt/what-is/service-oriented-architecture/>.
- [2] Sten Pittet. *Diferentes tipos de testes de software*. 2024. URL: <https://www.atlassian.com/br/continuous-delivery/software-testing/types-of-software-testing>.
- [3] Adhir Potdar. *Containerization: A Beginner's Guide to its Impact on Software Development*. 2023. URL: <https://bito.ai/blog/containerization-a-beginners-guide-to-its-impact-on-software-development>.
- [4] Luciano Rocha. *Reutilização de Software na Engenharia de Software: Conceitos, Benefícios e Desafios*. URL: <https://pt.linkedin.com/pulse/reutiliza%C3%A7%C3%A3o-de-software-na-engenharia-conceitos-e-gomes-rocha-odxxf>. 29/12/2023.
- [5] *The Benefits of Automation Testing*. URL: <https://smartbear.com/learn/automated-testing/>.
- [6] *What is End-to-end (E2E) Testing?* URL: <https://smartbear.com/learn/automated-testing/what-is-end-to-end-testing/>.